Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251172**
Nama Lengkap	**Ang, Joshua Alexander Hartono**
Minggu ke / Materi	**06 / Struktur Kontrol Perulangan**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Definisi Perulangan

Dalam pemrograman, jalannya sebuah program dapat diatur dengan berbagai macam cara. Cara ini dapat berupa sekuensial, perulangan, maupun kombinasi dari keduanya. Hal ini penting untuk dipahami karena sangat menentukan bagaimana sebuah program akan memproses perintah yang diberikan mulai dari awal sampai dengan akhir. Salah satunya adalah perulangan, yaitu proses untuk menjalankan suatu perintah secara berulang-ulang sesuai dengan kondisi yang telah diberikan. Selain itu, perulangan juga dapat digunakan untuk melakukan sesuatu secara bertahap. Pada tahap ini program yang ditulis memiliki langkah yang sama dan perlu dilakukan berkali-kali. Dalam hal ini perulangan membantu programmer membuat kode menjadi lebih singkat, efisien dan mudah untuk dikelola. Terakhir, perulangan juga berperan saat program harus mengakses data yang tersimpan dalam struktur data. Beberapa contoh struktur data yang sering digunakan adalah List, Tuple, Queue, Stack, dan beberapa struktur data lainnya. Dengan menggunakan perulangan, program dapat mengambil atau memproses setiap elemen yang ada satu per satu. Pada bahasa pemrograman Python, terdapat beberapa cara menggunakan perulangan yaitu dengan perulangan for, perulangan while, dan perulangan rekursif.

Perulangan For

Pada bahasa pemrograman Python, perulangan dapat dilakukan dengan menggunakan dua cara yaitu perulangan for dan perulangan while. Perulangan for biasanya digunakan ketika ada kondisi jumlah perulangan sudah diketahui sejak awal. Contohnya ketika program harus membaca 10 teks file secara berurutan. Walaupun setiap file teks memiliki isi yang berbeda, langkah untuk membaca file tetaplah sama. Pembacaan akan dilakukan dari file pertama dilanjutkan dengan file kedua, ketiga dan seterusnya sampai dengan file kesepuluh. Dengan menggunakan for, proses tersebut dapat dilakukan dengan efisien tanpa harus menuliskan kode yang sama berkali-kali.

Selain itu, perulangan for juga sering digunakan ketika program melakukan operasi yang sama pada suatu rentang nilai tertentu. Contohnya ketika program perlu untuk menghitung jumlah dari seratus bilangan pertama. Dalam hal ini, proses yang akan dilakukan adalah menjumlahkan angka secara berurutuan mulai dari angka 1, 2, 3, dan seterusnya sampai dengan angka ke 100. Walaupun nilai yang dijumlahkan terus bertambah, namun langkah yang dilakukan program akan tetap sama yaitu melakukan operasi penjumlahan secara berulang.

Dalam Python, perulangan for pada suatu rentang nilai dapat ditulis menggunakan bantuan fungsi range(). Fungsi ini digunakan untuk menghasilkan sekumpulan angka dalam rentang tertentu yang akan diproses oleh perulangan. Bentuk range yang paling sering digunakan adalah range(stop) akan menghasilkan angka mulai dari 0 sampai dengan stop – 1. Misalnya range(6) akan menghasilkan angka 0, 1, 2, 3, 4, dan 5.

Selain bentuk tersebut, fungsi range() juga memiliki bentuk yaitu range(start, stop, step). Pada bentuk ini, programmer dapat menentukan angka awal perulangan, batas akhir, serta jarak kenaikan setiap langkahnya. Nilai start berfungsi untuk menunjukkan awal angka, stop menunjukkan batas akhir, sedangkan step akan menunjukkan seberapa besar perubahan nilai setiap kali perulangan berjalan. Jika nilai step tidak ditentukan, maka Python akan menggunakan nilai 1. Berikut adalah contoh program untuk menampilkan angka dari 1 sampai dengan 100 menggunakan perulangan for di Python.

```
for i in range(1, 101):
    print(i)
```

Gambar 6.1: Contoh penggunaan for untuk menampilkan angka 1 sampai 100. *Sumber:* https://sl1nk.com/2zC4o

Pada program tersebut, perulangan dimulai dari angka 1 sebagai nilai awal (start) dan berakhir sebelum angka 101 (stop-1) dengan default step 1. Hal ini berarti angka akan ditampilkan dari 1 sampai dengan angka ke 100. Selain itu, terdapat i yang berfungsi sebagai counter, yaitu variabel yang akan menyimpan nilai setiap kali perulangan dilakukan.

Dalam beberapa kondisi, nilai dari counter tidak digunakan di dalam program. Jika counter tidak diperlukan, Python menyediakan cara yang lebih sederhana yaitu dengan menggunakan tanda garis bawah (_). Simbol ini biasa digunakan untuk menunjukkan bahwa nilai dari variabel tidak akan dipakai di dalam program. Contoh:

```
for _ in range(1, 101):
    print('Hello World')
```

Gambar 6.2: Contoh penggunaan garis bawah pada perulangan for. *Sumber:* https://sl1nk.com/2zC4o

Pada program tersebut, program akan menampilkan tulisan "Hello World" sebanyak 100 kali. Walaupun perulangan berjalan sebanyak 100 kali, tapi nilai dari counter tidak digunakan untuk proses apapun hanya untuk mengulangi suatu perintah beberapa kali.

Selain menggunakan langkah positif, fungsi range juga dapat digunakan dengan cara step negatif. Step negatif digunakan ketika perulangan dilakukan secara menurun dari angka yang lebih besar menuju angkan yang lebih kecil. Contoh:

```
for i in range(2, 101, 2):
    print(i)
```

Gambar 6.3: Program untuk menampilkan angka genap dari 2 sampai 100. *Sumber:* https://sl1nk.com/2zC4o

Program tersebut berfungsi untuk memunculkan perulangan mulai dari angka 2 dan berakhir sebelum angka 101 dengan jarak atau selisih 2. Sehingga program ini akan menampilkan angka mulai dari 2, 4, 6, 8, 10, dan seterusnya sampai dengan 100.

```
for i in range(100, 1, -2):
    print(i)
```

Gambar 6.4: Program untuk menampilkan angka genap dari 100 sampai 2. *Sumber:* https://sl1nk.com/2zC4o

Program tersebut akan memberikan output berupa angka 100, 98, 96, 94, 92, dan seterusnya sampai dengan angka 2. Hal ini dapat terjadi karena nilai step yang digunakan adalah -2, sehingga nilai dari counter akan terus berkurang sebanyak 2 setiap kali perulangan terjadi.

Bentuk Perulangan While

Perulangan While biasanya digunakan ketika jumlah perulangan belum diketahui sejak awal. Artinya program akan terus melakukan perulangan selama kondisi yang ditetapkan masih terpenuhi. Ketika kondisi tersebut tidak terpenuhi maka perulangan akan berhenti. Sehingga perulangan while sering digunakan pada program yang bergantung pada kondisi tertentu.

Bentuk yang sering digunakan pada perulangan while di Python dapat ditulis sebagai berikut:

```
<start>
while <stop>:
    operation
    operation
    <step (optional)>
```

Gambar 6.5: Bentuk perulangan while secara umum. *Sumber:* https://sl1nk.com/2zC4o

Pada bentuk tersebut, bagian start berfungsi untuk menentukan nilai awal dari suatu variabel. Kemudian pada bagian while terdapat kondisi yang akan diperiksa setiap kali perulangan berjalan. Jika kondisi bernilai true maka operasi dalam perulangan akan dijalankan. Setelah itu biasanya terdapat step yaitu perubahan nilai variabel yang dapat mempengaruhi perulangan.

```
1  bilangan = 0
2  genap = False
3  while genap == False:
4      bilangan = int(input('Masukkan bilangan genap: '))
5      if bilangan % 2 == 0:
6          genap = True
7  print(bilangan, 'yang anda masukkan adalah bilangan genap')
```

Gambar 6.6: Contoh perulangan while. *Sumber:* https://sl1nk.com/2zC4o

Penggunaan Break dan Continue

Dalam Python, perulangan dapat dikontrol menggunakan perintah break dan continue. Kedua perintah ini berfungsi untuk mengatur perulangan agar sesuai dengan kebutuhan program. Perintah break biasa digunakan untuk menghentikan perulangan. Sedangkan perintah continue berfungsi untuk melewati suatu tahap perulangan dan langsung lanjut ke tahap berikutnya. Contoh:

```
for i in range(1, 11):
    if i == 5:
        break
    else:
        print(i)
print('Selesai')
```

Gambar 6.7: Contoh penggunaan break pada perulangan. *Sumber:* https://sl1nk.com/2zC4o

Pada program tersebut, perulangan dirancang untuk bekerja dari 1 sampai ke 10. Namun ketika variabel mencapai angka 5, kondisi i == 5 bernilai True. Sehingga perintah break akan berjalan yang mengakibatkan program berhenti. Dengan demikian program hanya akan menghasilkan angka 1 sampai 4 dan diakhiri dengan selesai.

```
for i in range(1, 11):
    if i == 6:
        continue
    else:
        print(i)
print('Selesai')
```

Gambar 6.8: Contoh penggunaan continue pada perulangan. *Sumber:* https://sl1nk.com/2zC4o

Pada program tersebut akan menampilkan angka 1 sampai 10, tetapi angka 6 tidak akan ditampilkan. Hal ini karena ketika i == 6 maka program akan menjalankan perintah continue sehingga proses print(i) dilewati. Setelah itu program akan lanjut ke iterasi berikutnya.

```
C:\ProgramData\Microsoft\Win
(base) PS C:\Users\yuanl\Documents\Github\modul-alpro-python
1
2
3
4
5
7
8
9
10
Selesai
```

Gambar 6.9: Penggunaan continue untuk melewatkan iterasi saat counter(i) bernilai 6. *Sumber:* https://sl1nk.com/2zC4o

onversi dari Bentuk for Menjadi Bentuk while

Perulangan for pada Python pada dasarnya dapat diubah menjadi bentuk while. Hal ini karena kedua perulangan tersebut memiliki nilai awal, kondisi akhir, dan step agar perulangan dapat berjalan dengan benar.

```
for i in range(1, 11):
    print(i)
```

Gambar 6.10: Perulangan for untuk menampilkan 1 sampai 10. *Sumber:* https://sl1nk.com/2zC4o

Perulangan tersebut dimulai dari 1, berakhir di 10, dan memiliki langkah 1 per iterasinya. Jika dikonversi menjadi while maka nilai awal akan ditentukan terlebih dahulu, kemudian kondisi akhir akan diperiksa pada while dan nilai variabel diperbaharui setiap perulangan.

```
i = 1                //awal
while i <= 10:       //kondisi akhir
    print(i)
    i = i + 1        //step
```

Gambar 6.11: Perulangan while untuk menampilkan 1 sampai 10. *Sumber:* https://sl1nk.com/2zC4o

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github : https://github.com/angjoshua-sudo/Prak-Alpro-2026.git

SOAL 1

Source code:

```python
def perkalian(angka1, angka2):
    akhir = 0
    i = 1
    teks = ""

    while angka1 >= i:
        akhir = akhir + angka2
        teks = teks + str(angka2)

        if angka1 > i:
            teks = teks + " + "

        i += 1
    print (f"{angka1} x {angka2} = {teks} = {akhir}")

perkalian(6,5)
perkalian(7,10)
```

Penjelasan:

Program diawali dengan fungsi **perkalian** dengan dua parameter yaitu **angka1** dan **angka2**. Fungsi ini bertujuan untuk menghitung hasil perkalian dengan menggunakan penjumlahan berulang. Pada awal fungsi terdapat variabel **ahkir** = 0 untuk menyimpan hasil dari perhitungan. Selanjutnya ada variabel **i = 1** sebagai counter dalam proses perulangan. Adapun variabel **teks** yang diisi dengan string kosong dengan tujuan untuk menyimpan bentuk penjumlahan.

Selanjutnya terdapat **perulangan while** dengan kondisi selama i masih lebih kecil atau sama dengan **angka1**. Sehingga perulangan akan terus dijalankan sebanyak **angka1**. Di dalam perulangan tersebut, variabel **akhir** ditambahkan dengan **angka2** setiap kali terjadi perulangan sehingga proses perkalian dilakukan dengan cara menjumlahkan **angka2** secara berulang. Selainn itu, **angka2** juga ditambahkan ke dalam variabel **teks** dalam bentuk string agar dapat ditampilkan sebagai baris penjumlahan. Selanjutnya ada kondisi **if** yang berfungsi untuk mengecek apakah perulangan masih akan berlanjut. Jika masih berlanjut maka tanda dan spasinya **“ + “** akan

ditambahkan pada variabel **teks** sehingga tampilannya dapat menjadi **{angka2} + {angka2} + {angka2} + … .**

Setelah perulangan selesai, program akan menampilkan hasil menggunakan fungsi **print**. Pada bagian akhir program dilakukan pemanggilan fungsi **perkalian(6,5)** dan **perkalian(7,10)** untuk menjalankan fungsi.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu6/soal1.
py
6 x 5 = 5 + 5 + 5 + 5 + 5 + 5 + 5 = 30
7 x 10 = 10 + 10 + 10 + 10 + 10 + 10 + 10 = 70
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github>
```

SOAL 2

Source code:

```python
def ganjil(bawah, atas):
    cek_bawah = bawah
    teks = ""

    if bawah < atas:

        if cek_bawah % 2 == 0:
            cek_bawah = cek_bawah + 1

        for i in range(cek_bawah, atas +1, 2):
            teks = teks + str(i)
            if i + 2 <= atas:
                teks = teks + ", "
        print(f"bawah = {bawah}, atas = {atas}. Karena bawah < atas, berati dari kecil ke besar, maka hasilnya adalah: {teks}.")

    else:

        if cek_bawah % 2 == 0:
            cek_bawah = cek_bawah - 1

        for i in range(cek_bawah, atas - 1, -2):
            teks = teks + str(i)
            if i - 2 >= atas:
                teks = teks + ", "
        print(f"bawah = {bawah}, atas = {atas}. Karena bawah > atas, berati dari besar ke kecil, maka hasilnya adalah: {teks}.")


ganjil(10,30)
ganjil(97,82)
```

Penjelasan:

Program diawali denga fungsi **ganjil** yang memiliki dua parameter yaitu **bawah** dan **atas**. Parameter **bawah** menunjukkan batas awal bilangan, sedangkan **atas** menujukkan batas akhir bilangan. Pada awal fungsi dibuat variabel **cek_bawah** yang diisi dengan nilai dari variabel **bawah**. Variabel **cek_bawah** digunakan untuk mengecek apakah nilai batas **bawah** merupakan bilangan

ganjil atau genap. Selain itu variabel **teks** berisi string kosong untuk menyimpan deret bilangan ganjil.

Selanjutnya ada program menggunakan **percabangan if** untuk mengecek apakah nilai **bawah lebih kecil dari atas**. Jika kondisi tepenuhi maka deret billangan akan menampilkan angka **dari kecil ke besar**. Pada bagian awal, program akan mengecek apakah **cek_bawah** merupakan sebuah angka genap atau tidak menggunakan modulus % 2. Jika hasil sama dengan 0 maka bilangan tersebut genap dan perlu ditambah dengan satu agar menjadi bilangan ganjil. Setelah itu, program akan menjalankan **perulangan** for menggunakan fungsi **range** dengan step 2 agar hanya bilangan ganjil yang terambil. Setelahnya setiap bilangan ganjil akan masuk ke variabel **teks** dalam bentuk string. Kemudian ada kondisi **if** yang berfungsi untuk menambahkan tanda koma sebagai pemisah antar bilangan selama masih ada bilangan berikutnya dalam deret. Setelah perulangan selesai, program akan menampilkan hasilnya mengggunakan fungsi **print** yang menjelaskan nilai batas bawah, batas atas, serta deret bilangan ganjil yang dihasilkan

Jika kondisi **bawah < atas** tidak terpenuhi, maka program akan menjalankan bagian **else**. Pada bagian ini berati **bawah lebih besar dari atas**, sehingga deret bilangan ganjil akan ditampilkan **dari besar ke kecil**. Program Kembali mengecek apakah **cek_bawah** merupakan bilangan genap. Jika genap maka nilainya dikurangi 1 agar menjadi balagan ganjil. Selanjutnya **perulangan for** dengan fungsi **range** yang dimulai **cek_bawah** hingga melewati atas dengan step **-2** agar bilangan berkurang 2 setiap terjadi perulangan. Setiap balagan ganjil kemudian akan ditambahkan ke dalam **teks**, kemudian tanda koma akan ditambahkan dengan sebagai pemisah jika masih ada angka berikutnya. Setelah proses perulangan selesai, program akan menampilkan deret bilangan ganjil serta keterangan bahwa deret bilanga tersebut ditampilkan dari besar ke kecil.

Pada bagian akhir, fungsi ganjil yaitu **ganjil(10,30)** dan **ganjil(97,82)** dipanggil untuk menjalankan fungsi tersebut.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu6/soal2.py
bawah = 10, atas = 30. Karena bawah < atas, berati dari kecil ke besar, maka hasilnya adalah: 11, 13, 15, 17, 19, 21, 23, 25, 27, 29.
bawah = 97, atas = 82. Karena bawah > atas, berati dari besar ke kecil, maka hasilnya adalah: 97, 95, 93, 91, 89, 87, 85, 83.
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github>
```

SOAL 3

Source code:

```python
import math
print("Program penghitung IPS Mahasiswa")
jumlah_matkul = int(input("Berapa jumlah mata kuliah? "))

i = 1
total = 0
error=False

while i <= jumlah_matkul:
    huruf = input(f"Nilai Mk {i}: ")
    i = i + 1

    if huruf == "A":
        nilai = 4
    elif huruf == "B":
        nilai = 3
    elif huruf == "C":
        nilai = 2
    elif huruf == "D":
        nilai = 1
    else:
        print("Nilai harus A, B, C, atau D!")
        error = True
        break

    total += nilai*3

if not error:
    ips = total / (jumlah_matkul*3)

    print(f"Nilai IPS anda semester ini {round(ips,2)}")
```

Penjelasan:

Program ini bertujuan untuk mehitung **Indeks Prestasi Mahasiswa (IPS).** Program diawali dengan menampilkan judul **"Program penghitung IPS Mahasiswa"** agar user menegtaui fungsi dari program. Selanjutnya program akan meminta user memberikan input berupa angka untuk **jumlah mata kuliah** yang akan diambil semester tersebut.

Setelah itu, terdapat variabel **i** yang bernilai 1 sebagai counter dalam proses perulangan. Adapun variabel 0 untuk menyimpan jumlah nilai yang diperoleh. Program kemudian menggunakan **perulangan while** untuk meminta pengguna memasukkan **nilai huruf** untuk setiap mata kuliah dengan jumlah mata kuliah yang dimasukkan.

Di dalam perulangan tersebut digunakan **if, elif,** dan **else** untuk menentukan bobot nilai dari setiap huruf yang dimasukkan. Jika pengguna memasukkan nilai **A**, maka bobotnya adalah **4**. Jika

nilai **B**, bobotnya **3**, nilai **C** bobotnya **2**, dan nilai **D** bobotnya **1**. Setiap nilai kemudian dikalikan dengan 3 SKS, karena diasumsikan setiap mata kuliah memiliki jumlah jumlah **SKS** yang sama, yaitu 3. Hasil perkalian tersebut kemudian ditambahkan ke dalamm variabel **total**. Setelah seluruh nilai mata mata kuliah dimasukkan dan diproses, program akan menghitung **IPS** dengan rumus **total nilai dibagi dengan SKS**, yaitu jumlah mata kuliah dikalikan 3. Hasil tersebut kemudian ditampilkan kepada pengguna sebagai **nilai IPS semester**, dan dibulatkan hingga dua angka dibelakang koma.

Jika pengguna memasukkan nilai selain **A, B, C,** atau **D**, maka program akan menampilkan kesalahan sehingga user mengetahui bahwa nilai yang dimasukkan tidak sesuai dengan ketentuan.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu6/soal3.
py
Program penghitung IPS Mahasiswa
Berapa jumlah mata kuliah? 4
Nilai Mk 1: pk
Nilai harus A, B, C, atau D!
```